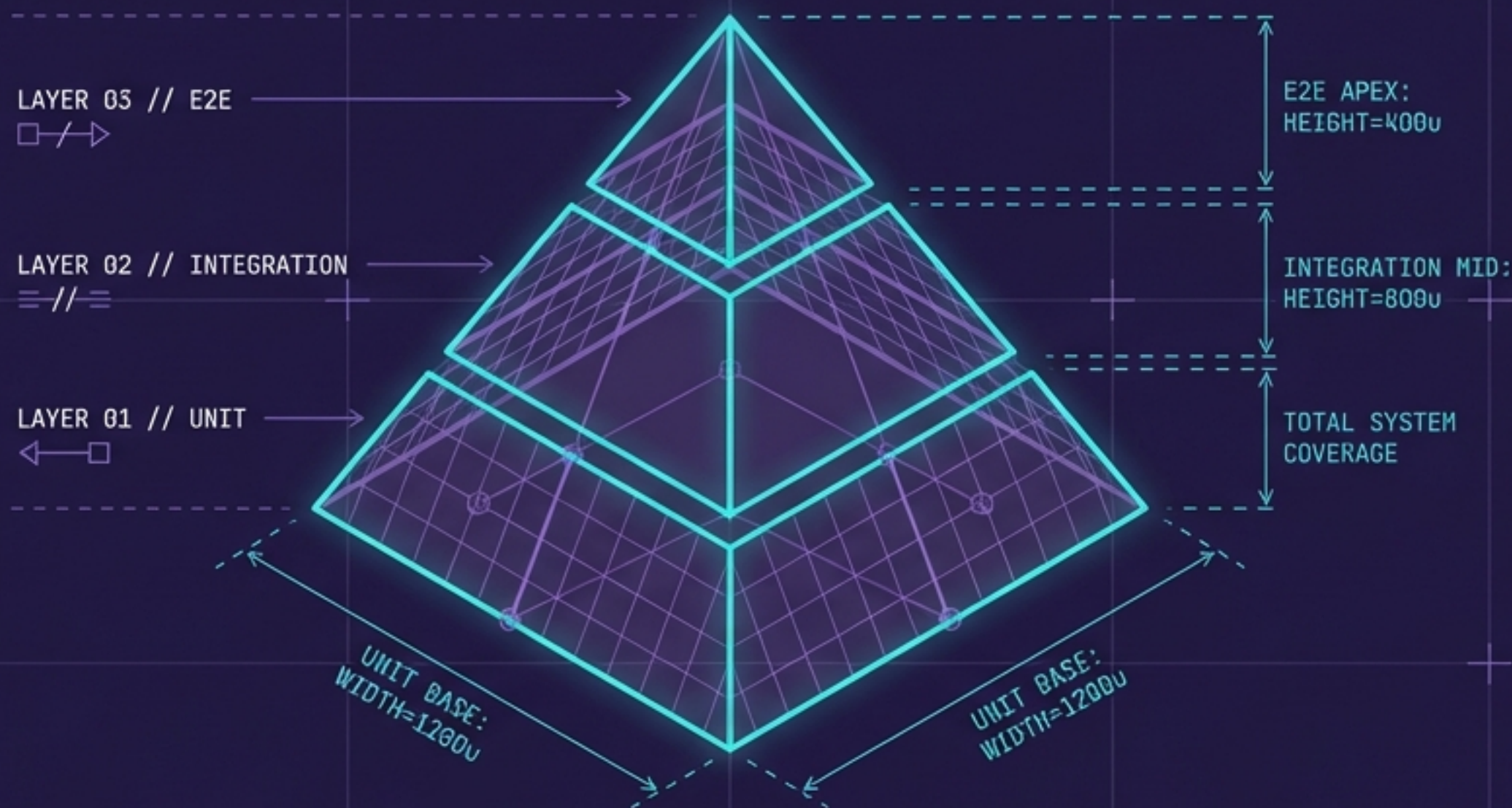


Estrategia Integral de Testing para Desarrolladores

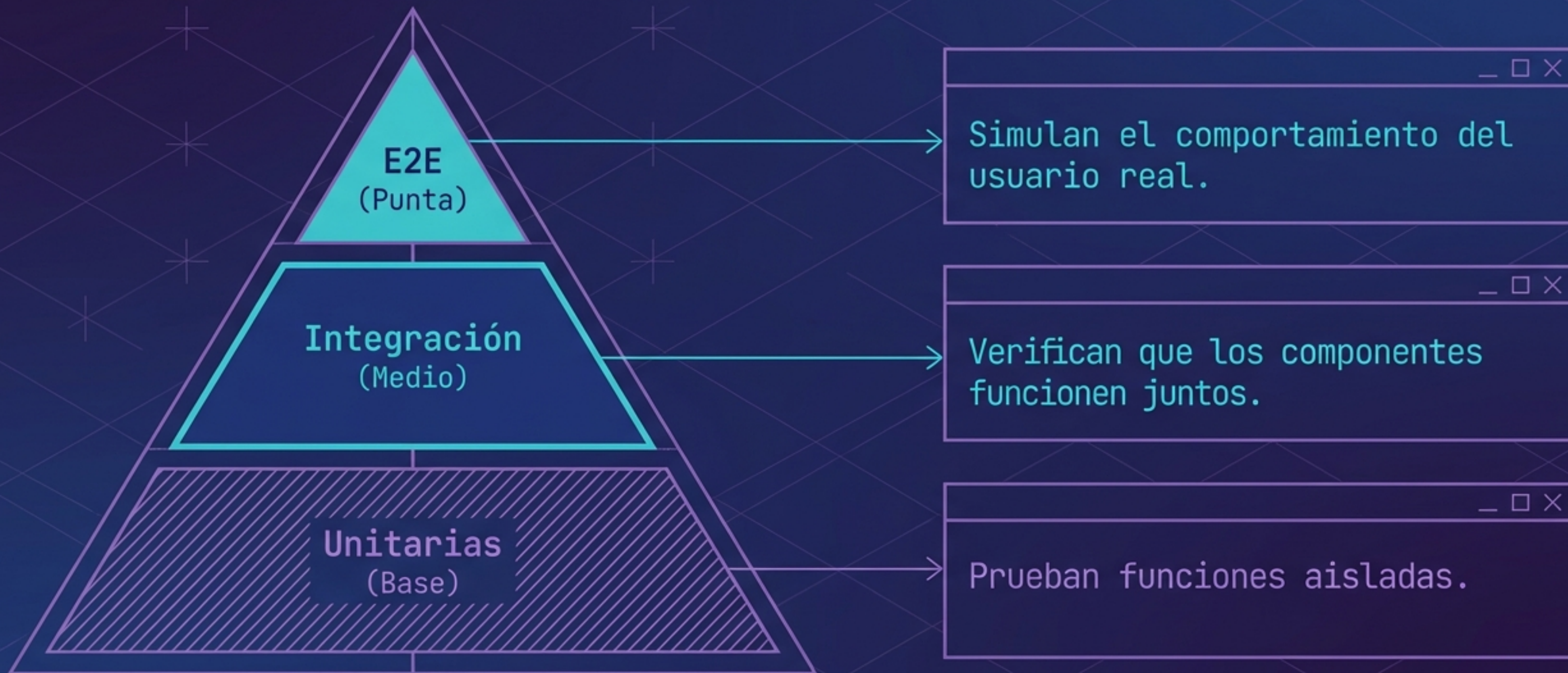
Fundamentos, Pirámide y Mejores Prácticas



Una guía técnica para implementar una estrategia de calidad basada en la pirámide de pruebas: Unitarias, de Integración y End-to-End (E2E).

Visión General: La Pirámide

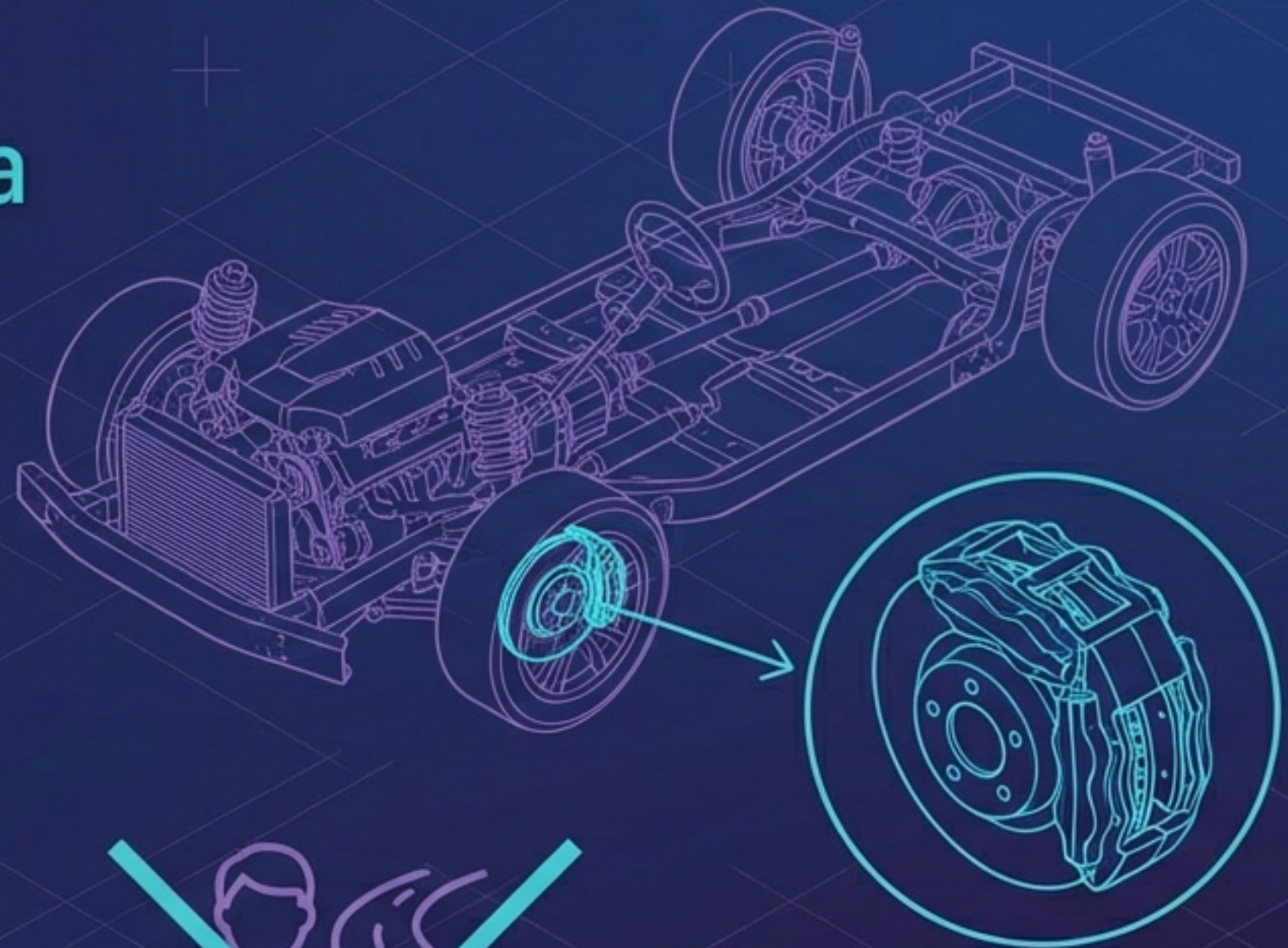
Tres niveles de defensa para una estrategia sólida.



Pruebas Unitarias: La Primera Línea de Defensa

Definición: Verifican una sola unidad de código (función, método o clase) de forma totalmente aislada.

Regla: No interactúan con bases de datos, APIs ni sistemas de archivos. Solo lógica pura.



Test Unitario:
¿Frenan los frenos?

NO verificamos si
el coche conduce.

¿Por qué son vitales los Unit Tests?

Razones clave para adoptar esta práctica fundamental en el desarrollo de software.



Velocidad

Son rápidas (segundos) y avisan inmediatamente si rompes algo.



Mejor Código

Obligan a escribir funciones pequeñas. Si es difícil de testear, hace demasiadas cosas.



Documentación Viva

Muestran exactamente cómo se usa una función y qué se espera de ella.



Refactorización Segura

Permiten mejorar código antiguo sin miedo a romper la funcionalidad.

Buenas Prácticas: Cómo escribir Unit Tests

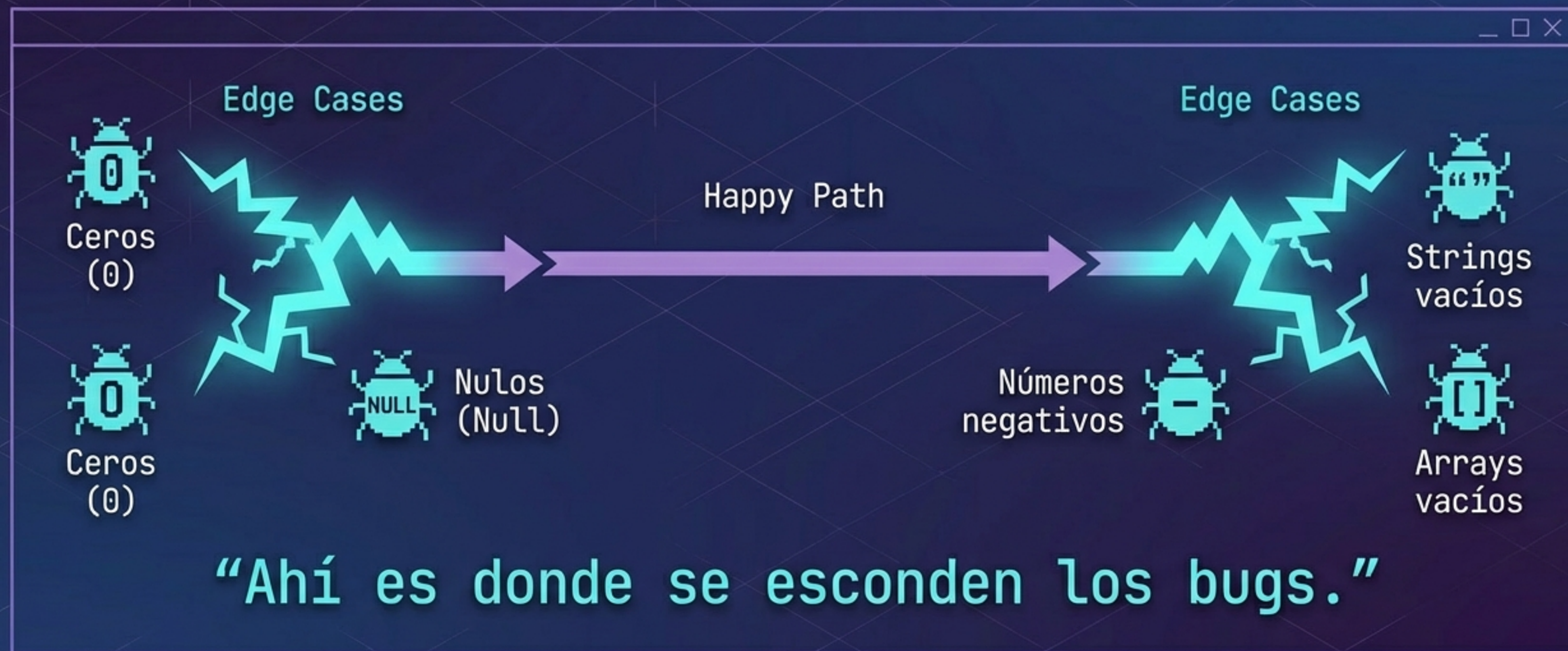
```
101 // Enfoque Único
102 1 test = 1 concepto. Mejor 5 tests pequeños que uno
103 gigante.
104
105 // Nombres Descriptivos
106 Evita `test1`. Usa nombres que expliquen el resultado
107 esperado.
108
109 // Aislamiento Total
110 No testes librerías externas (React, fechas). Testea
111 solo tu código.
112
113 // El Momento
114 Escribe el test antes o justo después de la función.
115 No esperes.
```



Enfócate en la Unidad:
Verificación Precisa

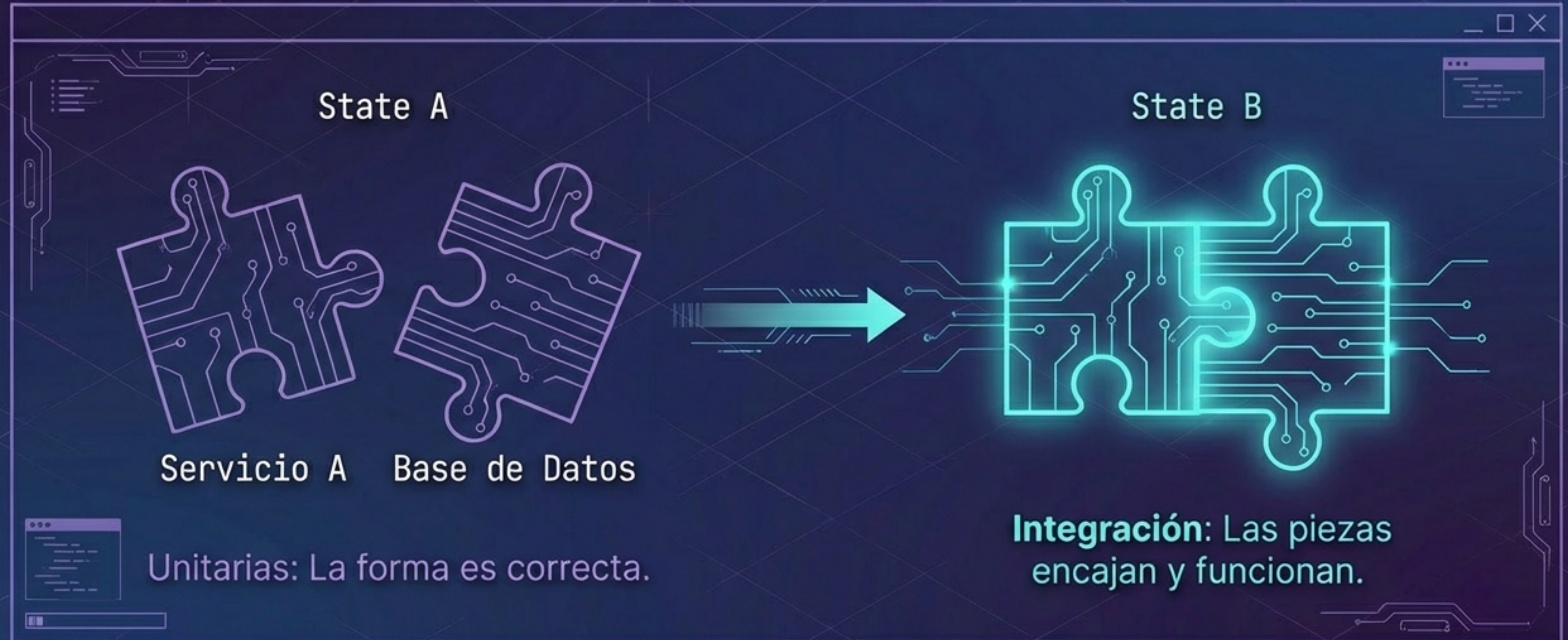
No solo el 'Camino Feliz': Caza los Edge Cases

La mayoría de los errores ocurren en los bordes.

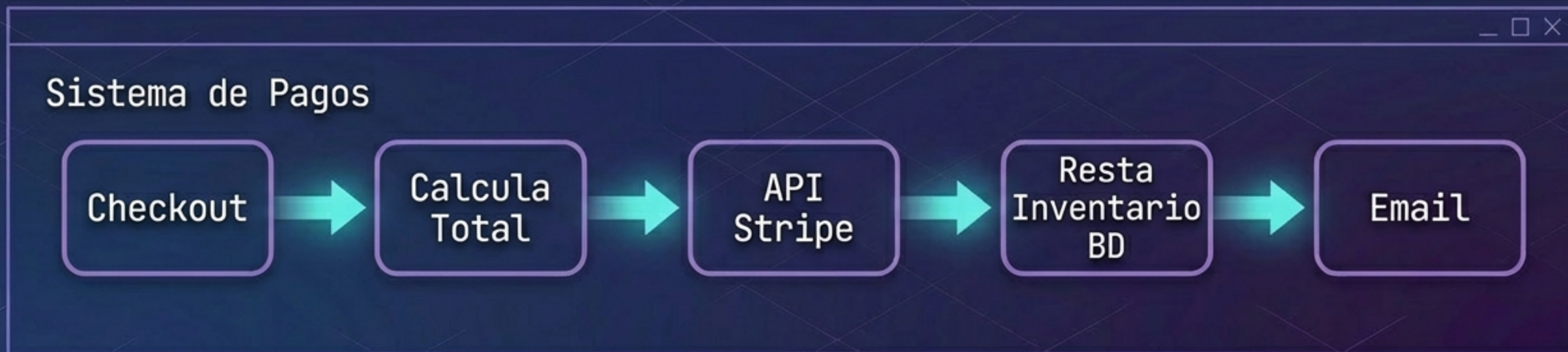
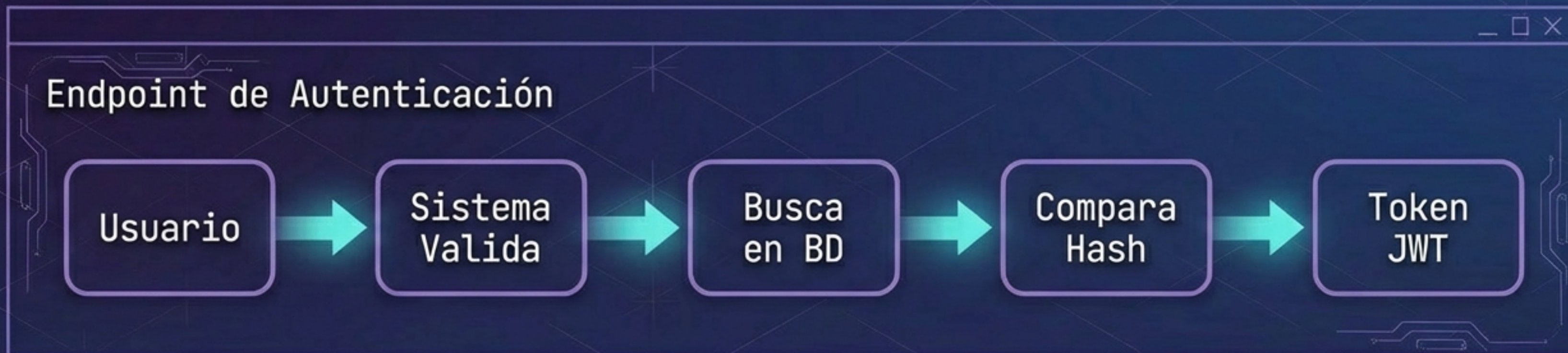


Pruebas de Integración: Conectando las Piezas

Verifican que diferentes partes del sistema (BD, APIs, servicios) funcionen correctamente cuando trabajan juntas.



Ejemplos de Flujos de Integración



Reglas de Oro para Integración

JetBrains Mono la: reglas crin oro de onconctor de reglas on para Integración , Inter es detenici como pruebas are conovable.



Base de Datos

NUNCA uses la DB de producción.
Usa Docker o una DB de test.



Estado Limpio (Reset)

Cada test inicia de cero. Borra datos o resetea la DB al finalizar.



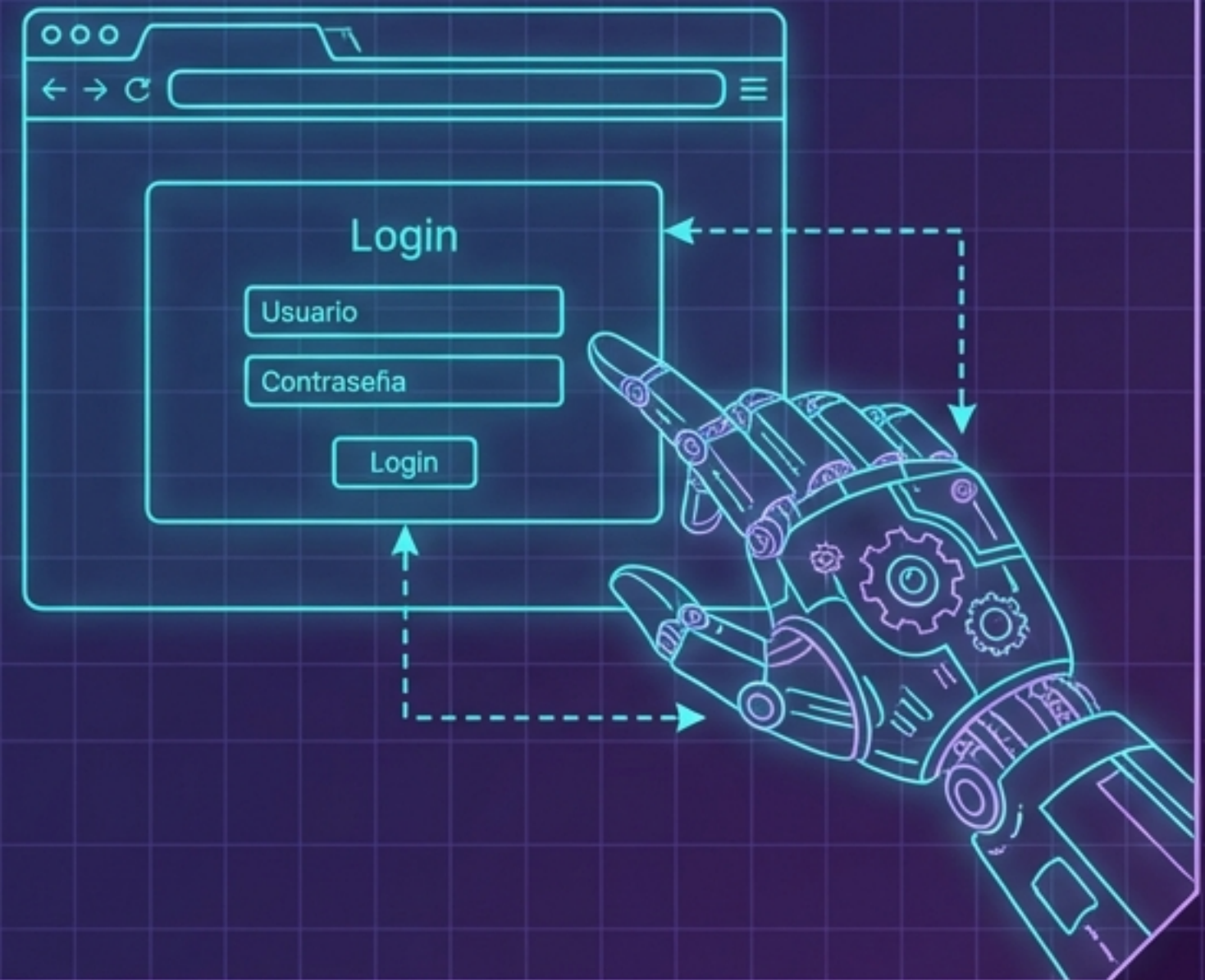
Datos Seguros

Usa datos realistas pero nunca sensibles (sin tarjetas reales).

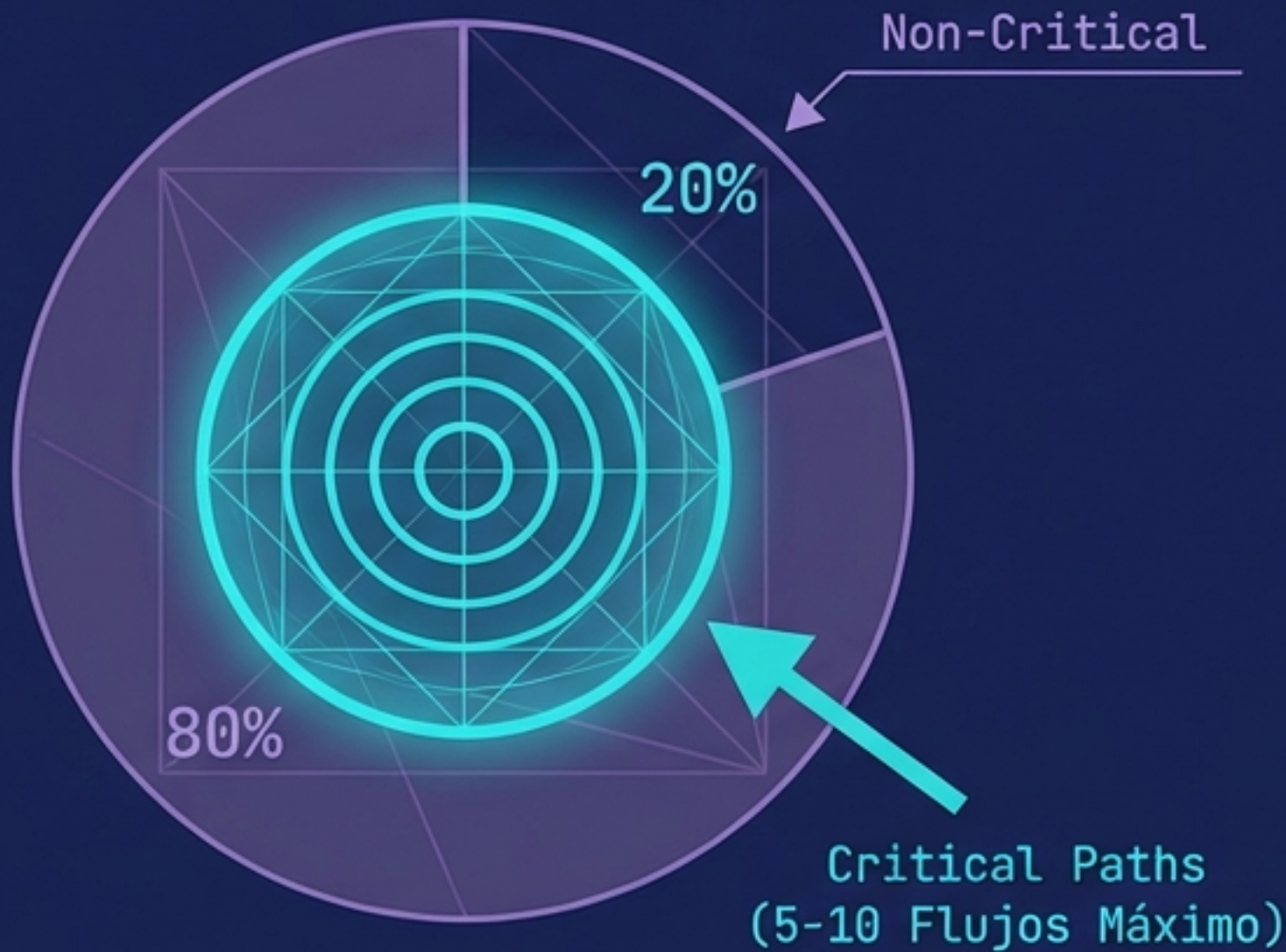
¡El entorno de pruebas debe ser desechable!

Pruebas End-to-End (E2E): La Simulación Final

1. **Definición:** Simulan al usuario real en un navegador.
2. **Concepto:** Son agnósticas al código (No importa si es React o Angular).
3. **Ventaja:** Aseguran flujos críticos antes del deploy.
4. **Desventaja:** Lentos y costosos de mantener.



Estrategia E2E: Menos es Más



- Autenticación (Login/Registro)
- Dinero (Compras/Suscripciones)
- Navegación Principal
- Operaciones Core del Negocio

No busques el 100% de cobertura en E2E. Es insostenible.

Herramientas Recomendadas para E2E

Cypress vs.

Playwright



```
{
```

- Más moderno y rápido
- Multi-navegador (Chromium, Firefox, WebKit)
- Multi-lenguaje (JS, TS, Python, C#, Java)
- Generador de código (Codegen)

```
}
```


Más allá de la Pirámide

Performance Tests



Velocidad y carga
(ej: 10k usuarios).

Security Tests



Vulnerabilidades y
ataques.

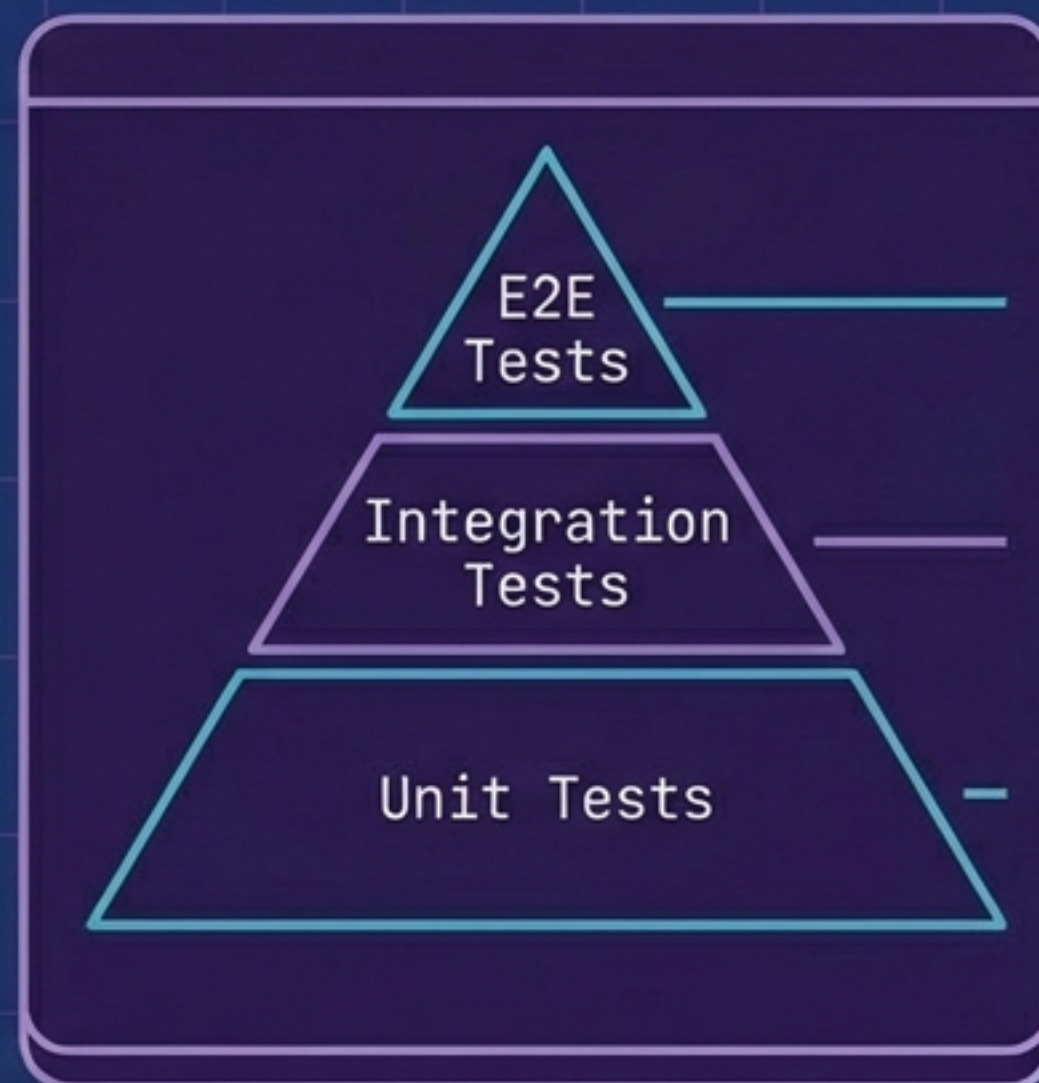
Regression Tests



Estabilidad ante
cambios nuevos.

Resumen: Tu Estrategia de Calidad

1. **Unit Tests** ✓ La base. Muchos tests, cubren lógica y casos borde. ✓
2. **Integration Tests** ✓ El medio. Comunicación entre servicios y BD. ✓
3. **E2E Tests** ✓ La punta. Flujos críticos de negocio.



Las pruebas no son opcionales. Te diferencian como profesional.

Construye con confianza.